

分布式系统

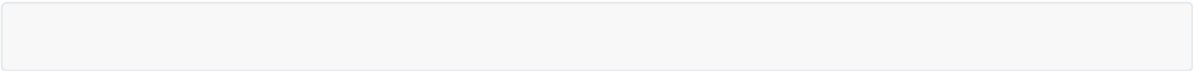
我们最初了解区块链的时候，很多人会形容这个区块链是一个“**分布式的不可篡改账本**”，这是一个很形象的说法，但是我认为更为准确的形容是“**所有节点共同维护的状态机**”。为什么分布式和区块链不能划等号呢？

两种常见的分布式模型

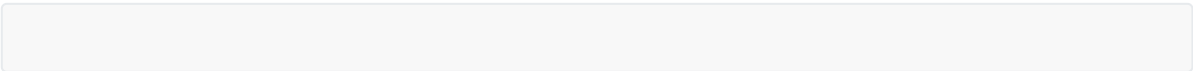
	中心化网络（Client-Server）	去中心化网络（P2P）
交互方式	用户只能和服务端交互	用户之间可以交互
优缺点	通讯速度快，客户端需要存储的信息量少，响应稳定；相对垄断	相对自由；通讯难度随着用户数量成指数级上升
涉及分布式问题	非拜占庭问题（只存在节点挂机情况）	拜占庭问题（存在干扰一致性的恶意节点）

图例

中心化网络的请求模型（响应与箭头方向相反）

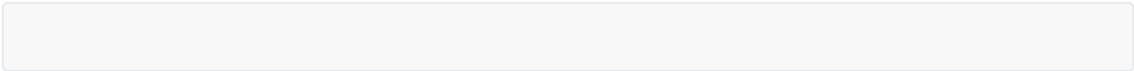


去中心化模型的请求模型



比较和联系

1. 对比上述两个图，我们很容易发现——去中心化的通信方式，每个节点需要保存的“通信信道”是超多的，因此这种模型无法适应大量的节点参与
2. 我们拿“**微信**”来举一个例子。微信的交流，虽然看起来是点对点的（C2C结构），但是实际上是用户请求云服务器来实现的信息交流的。（箭头方向是信息传递方向）



3. 以“种子网络”为例，在其发展初期存在的节点到其后期均被暴风影音等节点垄断。可以看出去中心化向中心化演变是一个自然的趋势。
4. 所以，我们通常的中心化分布式系统，是一个“**有超强中心节点的系统**”，在这个系统中只要服务器不挂机就是正常运行的，不存在恶意节点破坏一致性问题（因为只有服务器自己说了算）；而区块链涉及的分布式问题往往是有恶意节点存在的，这样的问题被称为“**拜占庭将军问题**”

拜占庭将军问题

背景

在网络上的节点分为恶意节点（目的是扰乱一致性）和故障节点（挂机）。拜占庭问题解决的是拥有恶意节点的问题；拥有故障节点的成为“非拜问题”

首先，我们来补充一下分布式账本的节点问题。拜占庭问题是由兰波特提出的，为了解决这个问题，采用了从理想化的“口头协议”-->解决网络延时的“PBFT算法”-->区块链使用的“PoW机制”

问题本身

拜占庭帝国派出多支军队进攻一个城池，这个城池十分坚固，一支部队无法攻克，但是多支军队一起攻打即可攻克。现在，将军中有叛徒，他的目的是使得忠诚的部队行动不一致（**叛徒的目的是为了扰乱系统的一致性**）。这个问题有解的前提是 $n(\text{总节点个数}) > 3m(\text{恶意节点个数})$ 。

问题解法

最初的解决方法——口头协议

这个解法使用的模型叫做**将军-副官模型**，在这个模型的要求下，所有的节点都可以是将军也都可以是副官（我们将第一个发送命令的节点叫做将军，其余的叫做副官，值得注意的是**副官与副官之间也可以通信**），所有节点需要遵守两个规则：

1. 忠诚的副官们会遵守同一个命令
2. 若将军忠诚，则所有的副官会遵守他的命令

下面我们看一下最基础（ $m=1$ 、 $n=4$ ）的情况

副官中出现叛徒：

1. 首先，我们将“将军”发布的命令称为A，即副官1，副官2，副官（叛徒）3收到的将军命令都是A
2. 这时，副官1会询问另外两个副官，得到副官2的“**将军给的是命令A**”，以及叛徒3给的“**将军给的命令是B**”（B是假消息）的回答
3. 因此，副官1按照少数服从多数原则，推断出3是叛徒，A是一致性命令。副官2同理

将军是叛徒：

1. 叛徒将军向副官1、2、3分别发送命令A、B、C（叛徒是为了扰乱一致性的）
2. 副官1接到A命令后会联系2、3，得到副官2的“**将军给的是命令B**”，以及副官3给的“**将军给的命令是C**”
3. 这时候副官1可以推断出将军是叛徒，同时放弃这一轮的命令（要求的第二条不成立），副官2、3同理

复杂情况（以m=2,n=7为例）

这种方法要注意以下两点：

- 1. **签名**：也就是一个人发布命令的时候需要加上自己的签名，转述别人的信息的时候需要加上他的签名（这是为了保证信息的唯一性），比如上文情况中我使用的“**副官说‘将军的命令是……’**”这一说法
- 2. 每个节点使用表格进行决策，这是一种递归嵌套的思想（下面表格展示的每行指的是节点的决策信息，纵列是节点给别人的应答），假设将军给出的命令是A

	V1给出的 应答	V2给出的 应答	V3给出的 应答	V4给出的 应答	V5给出的 应答	V6给出的 应答
V1		A	A	A	b	f
V2	A		A	A	c	g
V3	A	A		A	d	h
V4	A	A	A		e	i

上述的小写字母指的是随即命令，善意节点依照少数多数的原则可以推断出结果，在这里不展开叙述

通信次数

我们简单的计算一下上面两种模型的通讯次数发现，当存在四个节点中一个恶意节点时的通讯次数为**9**次，七个节点中存在两个恶意节点的通讯次数为**36**次，因此这种方法的算法复杂度随着节点数以指数级上升，是一种很不明智的算法。而下面要介绍的PBFT算法就将这种复杂度降到了多项式级。

用通讯次数换取安全性——PBFT（）算法

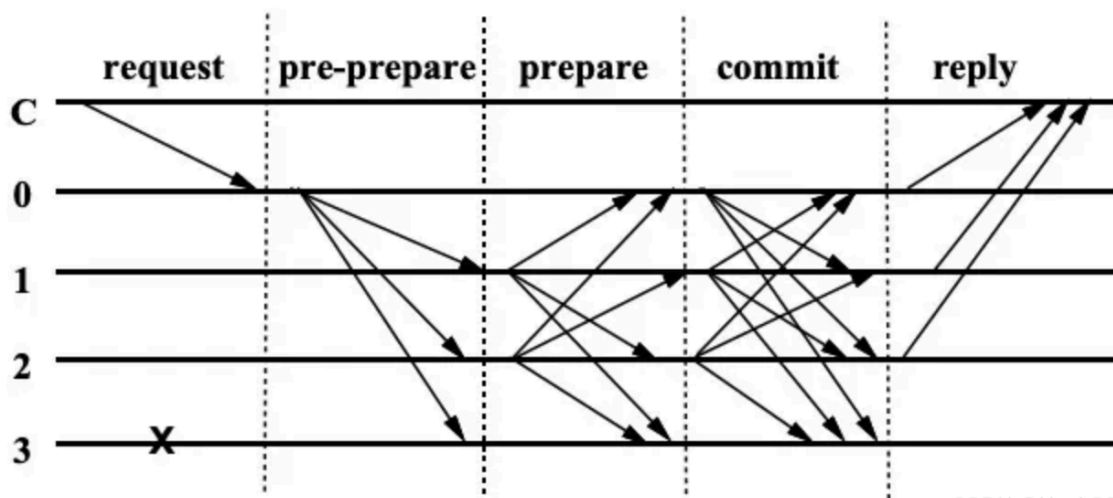
和“口头协议”一样，PBFT算法也是**使用通讯次数换取安全性**，这样的原理也决定了它只能在小范围中使用，实际生产中的应用场景往往是联盟链。

简介

PBFT 算法中节点只有两种角色，主节点（primary）和副本（replica），两种角色之间可以相互转换。两者之间的转换又引入了视图（view）的概念，视图在 PBFT 算法中起到逻辑时钟的作用。

算法实现流程

算法开始的时候，使用随机数或者一定的顺序得到主节点；首先客户端发送消息 m 给主节点 p，主节点就开始了 PBFT 三阶段协议，三个阶段分别是 预准备（pre-prepare），准备（prepare），提交（commit）。其中 pre-prepare 和 prepare 阶段最重要的任务是保证同一个主节点发出的请求在同一个视图（view）中的顺序是一致的，prepare 和 commit 阶段最重要的任务是保证请求在不同视图之间的顺序是一致的。算法的流程图如下：（在这个周期中，C指客户端节点，0指主节点，1、2指正常副本，3指掉线副本）



CSDN @Hock2024

编辑

- 主节点收到客户端发送来的消息后，构造 pre-prepare 消息结构体 $\langle \text{PRE-PREPARE}, v, n, d, m \rangle$ 广播到集群中的其它节点。（PRE-PREPARE 标识当前消息所处的协议阶段， v 表示当前视图编号， n 为主节点广播消息的一个唯一递增序号， d 为 m 的消息摘要， m 为客户端发来的消息）（这里的“摘要”也就是对于客户端消息进行的哈希压缩）
- 副本(backup) 收到主节点请求后，会对消息进行**检查**，检查通过会存储在本节点。当节点收到 $2f+1$ （包括自己）个相同的消息后，会进入 PREPARE 状态，广播消息 $\langle \text{PREPARE}, v, n, d, i \rangle$ ，其中 i 是本节点的编号。对消息的有效性有如下检查：（也就是满足超2/3节点达成共识）
 - 检查收到的消息体中摘要 d ，是否和自己对 m 生成的摘要一致，**确保消息的完整性**。
 - 检查 v 是否和当前视图 v 一致，**保证消息存在于同一周期**。
 - 检查序号 n 是否在水线 h 和 H 之间，**避免快速消耗可用序号**（ h 是当前的安全水线（safety waterline），表示系统中可以接受的最低序号； H 是当前的高水线（high waterline），表示系统中可以接受的最高序号； n 需要在这两个水线之间，即 $h \leq n \leq H$ ，主要是为了避免以下问题：**序号枯竭**：如果序号的使用没有控制在合理范围内，系统可能会迅速消耗掉所有可用的序号，这样会导致新的请求无法被处理，因为没有足够的序号可用。**性能问题**：不合适的序号管理可能会影响系统的性能，特别是当系统需要频繁地调整这些水线时，可能会造成额外的开销）。
 - 检查之前是否接收过相同序号 n 和 v ，但是不同摘要 d 的消息，**确保消息的唯一性**。
- 副本收到 $2f+1$ （包括自己）个一致的 PREPARE 消息后，会进入 COMMIT 阶段，并且广播消息 $\langle \text{COMMIT}, v, n, D(m), i \rangle$ 给集群中的其它节点。在收到 PREPARE 消息后，副本同样也会对消息进行有效性检查，检查的内容是上条的 1, 2, 3。
- 副本收到 $2f+1$ （包括自己）个一致的 COMMIT 个消息后执行 m 中包含的操作，其中，如果有多个 m 则按照序号 n 从小到大执行，执行完毕后发送执行成功的消息给客户端。

日志压缩（解决内存问题）

这种压缩方式采用的是“快照”的方法。Pbft 为常数个操作创建一次稳定检查点，比如每100个操作创建一次检查点，而这个检查点就是 checkpoint，当这个 checkpoint 得到集群中多数节点认可以后，就变成了**稳定检查点 stable checkpoint**。

- 当节点 i 生成 checkpoint 后会广播消息 $\langle \text{CHECKPOINT}, n, d, i \rangle$ 其中 n 是最后一次执行的消息序号， d 是 n 执行后的**状态机状态的摘要**。每个节点收到 $2f+1$ 个相同 n 和 d 的 checkpoint 消息以后，checkpoint 就变成了 stable checkpoint。

2. 同时删除本地序号小于等于 n 的消息。同时 checkpoint 还有一个提高水线 (water mark) 的作用, 当一个 stable checkpoint 被创建的时候, 水线 h 被修改为 stable checkpoint 的 n , 水线 H 为 $h + k$ 而 k 就是之前用到创建 checkpoint 的那个常数。

视图切换 (解决主机宕机)

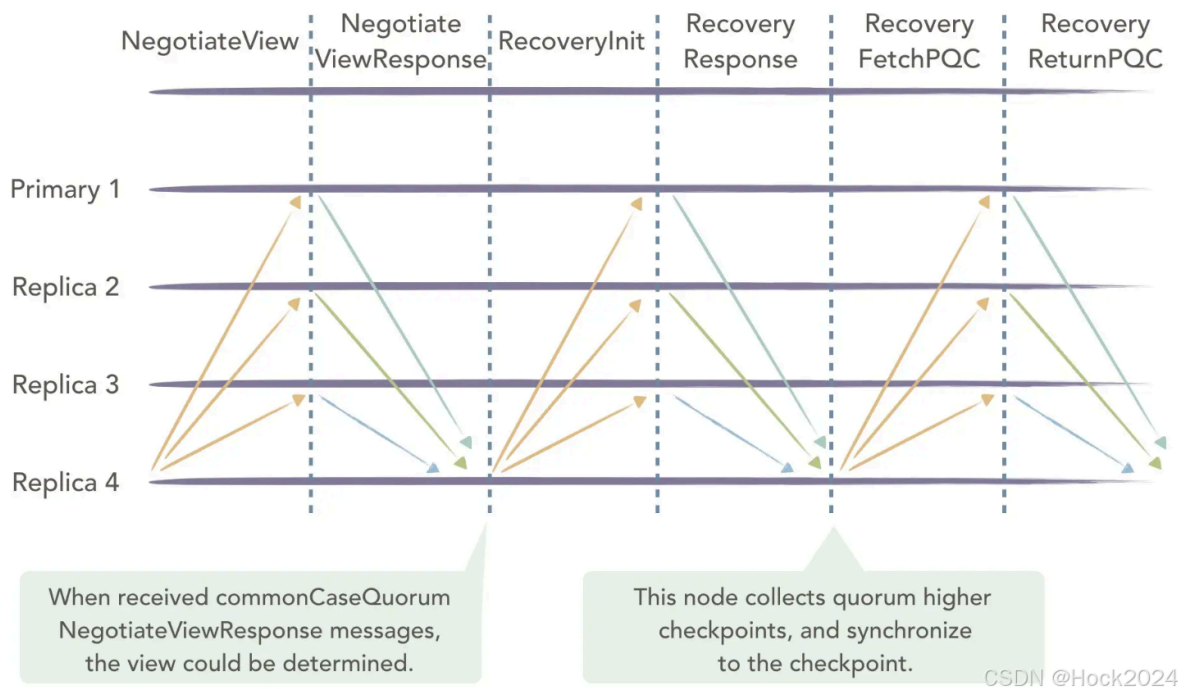
view-change 提供了一种当主节点宕机以后依然可以保证集群可用性的机制。view-change 通过计时器来进行切换, 避免副本长时间的等待请求。

1. 当副本收到请求时, 就启动一个计时器, 如果这个时候刚好有定时器在运行就重置 (reset) 定时器, 但是主节点宕机的时候, 副本 i 就会在当前视图 v 中超时, 这个时候副本 i 就会触发 view-change 的操作, 将视图切换为 $v+1$ 。副本 i 会停止接收除了 checkpoint, view-change 和 new view-change 以外的请求, 同时广播消息 $\langle \text{VIEW-CHANGE}, v+1, n, C, P, i \rangle$ 的消息到集群。
 1. n 是节点 i 知道的最后一个 stable checkpoint 的消息序号。
 2. C 是节点 i 保存的经过 $2f+1$ 个节点确认 stable checkpoint 消息的集合。
 3. P 是一个保存了 n 之后所有已经达到 prepared 状态消息的集合。
2. 当在视图 ($v+1$) 中的主节点 p_1 接收到 $2f$ 个有效的将视图变更为 $v+1$ 的消息以后, p_1 就会广播一条消息 $\langle \text{NEW-VIEW}, v+1, V, Q \rangle$
 1. V 是 p_1 收到的, 包括自己发送的 view-change 的消息集合。
 2. Q 是 PRE-PREPARE 状态的消息集合, 但是这个 PRE-PREPARE 消息是从 PREPARE 状态的消息转换过来的。
3. 从节点接收到 NEW-VIEW 消息后, 校验签名, V 和 Q 中的消息是否合法, 验证通过, 主节点和副本都进入视图 $v+1$ 。当 p_1 在接收到 $2f+1$ 个 VIEW-CHANGE 消息以后, 可以确定 stable checkpoint 之前的消息在视图切换的过程中不会丢, 但是当前检查点之后, 下一个检查点之前的已经 PREPARE 可能会被丢弃, 在视图切换到 $v+1$ 后, p_1 会把旧视图中已经 PREPARE 的消息变为 PRE-PREPARE 然后新广播。
 1. 如果集合 P 为空, 广播 $\langle \text{PRE-PREPARE}, v+1, n, \text{null} \rangle$, 接收节点就什么也不做。
 2. 如果集合 P 不为空, 广播 $\langle \text{PRE-PREPARE}, v+1, n, d \rangle$

总结一下, 在 view-change 中最为重要的就是 C , P , Q 三个消息的集合, C 确保了视图变更的时候, stable checkpoint 之前的状态安全。 P 确保了视图变更前, 已经 PREPARE 的消息的安全。 Q 确保了视图变更后 P 集合中的消息安全。回想一下 pre-prepare 和 prepare 阶段最重要的任务是保证, 同一个主节点发出的请求在同一个视图 (view) 中的顺序是一致的, 而在视图切换过程中的 C , P , Q 三个集合就是解决这个问题的。

主动恢复

集群在运行过程中, 可能出现网络抖动、磁盘故障等原因, 会导致部分节点的执行速度落后大多数节点, 而传统的 PBFT 拜占庭共识算法并没有实现主动恢复的功能, 因此需要添加主动恢复的功能才能参与后续的共识流程, 主动恢复会索取网络中其他节点的视图, 最新的区块高度等信息, 更新自身的状态, 最终与网络中其他节点的数据保持一致。在 Raft 中采用的方式是主节点记录每个跟随者提交的日志编号, 发送心跳包时携带额外信息的方式来保持同步, 在 Pbft 中采用了视图协商 (Negotiateview) 的机制来保持同步。一个节点落后太多, 这个时候它收到主节点发来的消息时, 对消息水线 (water mark) 的检查会失败, 这个时候计时器超时, 发送 view-change 的消息, 但是由于只有自己发起 view-change 达不到 $2f+1$ 个节点的数量, 本来正常运行的节点就退化为一个拜占庭节点, 尽管是非主观的原因, 为了尽可能保证集群的稳定性, 所以加入了视图协商 (Negotiateview) 机制。当一个节点多次 view-change 失败就触发 Negotiateview 同步集群数据, 流程如下;

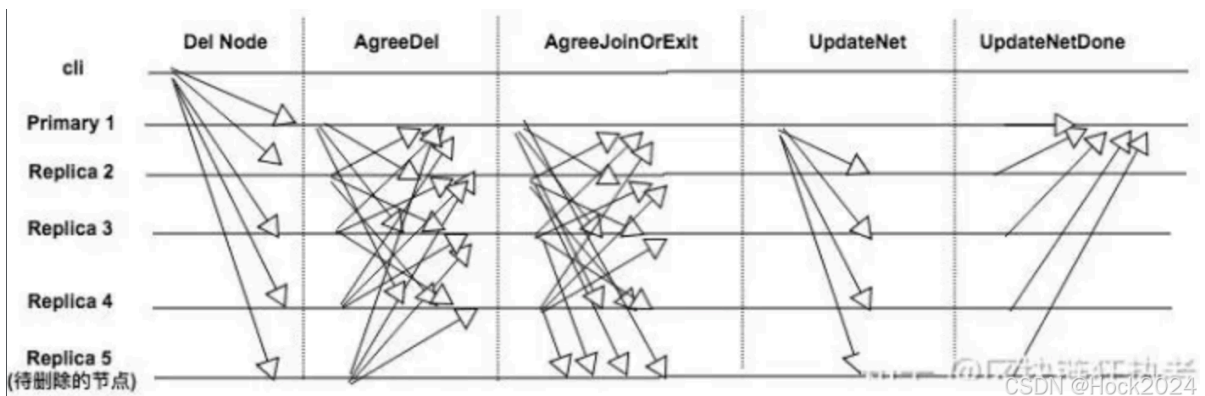


编辑

1. 新增节点 Replica 4 发起 NegotiateView 消息给其他节点;
2. 其余节点收到消息以后, 返回自己的视图信息, 节点ID, 节点总数N;
3. Replica 4 收到 $2f+1$ 个相同的消息后, 如果quorum个视图编号和自己不同, 则同步view和N;
4. Replica 4 同步完视图后, 发送 RevoeryToCheckpoint 的消息, 其中包含自身的 checkpoint 信息;
5. 其余节点收到 RevoeryToCheckpoint 后将自身最新的检查点信息返回给 Replica 4 ;
6. Replica 4 收到quorum个消息后, 更新自己的检查点到最新, 更新完成以后向正常节点索要 pset、qset和cset的信息 (即PBFT算法中pre-prepare阶段、prepare阶段和commit阶段的数据) 同步至全网最新状态;

增删节点 (解决成员加入或者删除的问题)

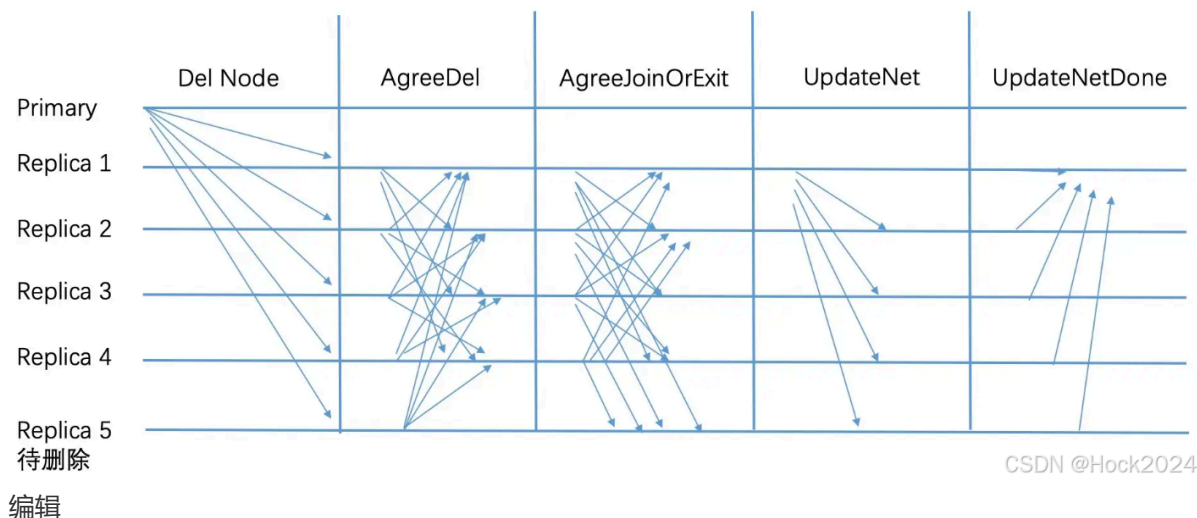
Replica 5 新节点加入的流程如下图所示;



编辑

1. 新节点启动以后, 向网络中其他节点建立连接并且发送 AddNode 消息;
2. 当集群中的节点收到 AddNode 消息后, 会广播 AgreeAdd 的消息;
3. 当一个节点收到 $2f+1$ 个 AgreeAdd 的消息后, 会发送 AgreeAdd 的消息给 `Replica 5`

4. `Replica 5` 会从收到的消息中，挑选一个节点同步数据，具体的过程在主动恢复中有说明，同步完成以后发送 `JoinNet`
 5. 当集群中其他节点收到 `JoinNet` 之后重新计算视图view，节点总数N，同时将PQC信息封装到 `AgreeJoinOrExit` 中广播
 6. 当收到 $2f+1$ 个有效的 `AgreeJoinOrExit` 后，新的主节点广播 `UpdateNet` 消息完成新增节点流程
- 删除节点的流程和新增节点的过程类似：



编辑

问题

Q：为什么 `PBFT` 算法需要三个阶段？ A：假如简化为两个阶段 `pre-prepare` 和 `prepare`，当一个节点A收到 $2f+1$ 个相同的 `prepare` 后执行请求，一部分节点B发生 `view-change`，在 `view-change` 的过程中是拒收 `prepare` 消息的，所以这一部分节点的状态机会少执行一个请求，当 `view-change` 切换成功后重放 `prepare` 消息，在重放的过程中，节点A也完成了 `view-change`，这个时候A就会面临重放的 `prepare` 已经执行过了，是否需要再次执行？会导致状态机出现二义性。

Q：view-change阶段集群会不可用么？ A：view-change阶段集群会出现短暂的不可用，一般在实践的时候都会实现一个缓冲区来减少影响，实现参考 [以太坊TXpool分析](#)。

Q：Pbft算法的时间复杂度？ A：Pbft算法的时间复杂度 $O(n^2)$ ，在 `prepare` 和 `commit` 阶段会将消息广播两次，一般而言，Pbft集群中的节点都不会超过100。

Pow算法——增加恶意节点的成本

由于上面的PBFT算法在节点增多到一定程度时就很难继续维持系统的一致性，因此在区块链网络的创建之初，创建者创造性的提出了PoW算法。PoW算法，使得每个需要广播公认信息的节点需要付出海量的代价，因此其原始意愿上就不倾向于去破坏系统的安全性，实现这个算法并不是依靠技术上的革新，反而像是商业模式上的变化。

PoW共识机制的特点：

1. 维持PoW算法的三个闭环元素：系统的安全性 获得奖励的激励 破坏系统的代价（下图展示了为什么PoW算法能够使得更多的人投入挖矿，这是一个由原始利益驱动的机制）

2. PoW算法必须与链式结构结合

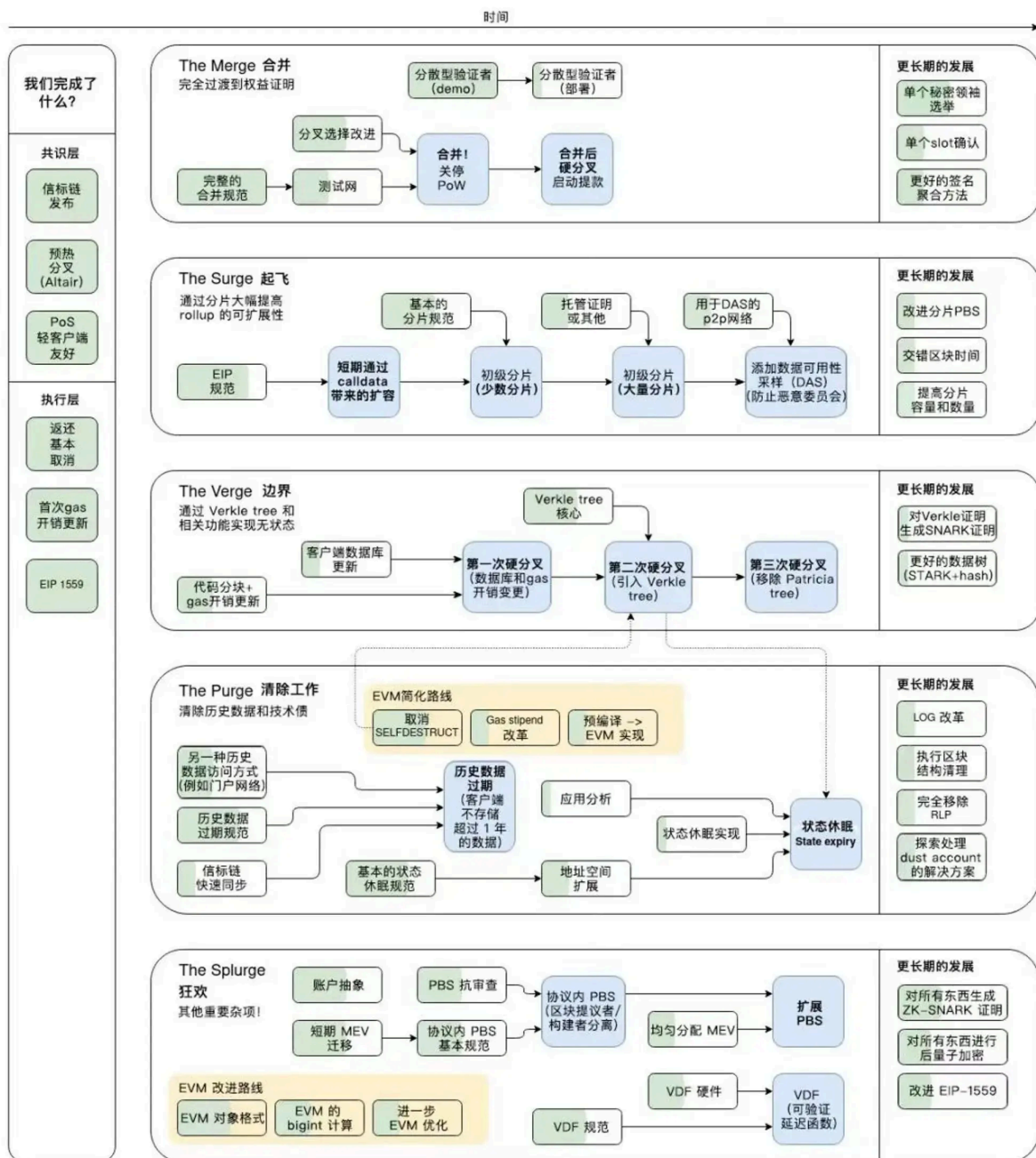
3. PoW算法使得共识达成只需要单轮通信，大大节省了通信成本

PoS算法

从PoW到PoS

以太坊在2014年开始了长达8年的pos研究之旅，2020年，以太坊上线了信标链（ beacon chain），并在这个链上尝试做一些pos的实验，2022年9月15日，将信标链与以太坊主链合并，至此完成了以太坊的升级，宣告了pow时代的结束。具体的流程分为三个阶段：

1. 在主链之外独立建设一条信标链（ beacon chain），在这条链上进行PoS实验
2. 合并阶段1，旧主链作为“执行层”，信标链作为“共识层”，融合之后进入以太坊PoS时代
3. 后续通过分片扩容提高rollup拓展性等等



来源: vitalik.eth 翻译: SETH 2024

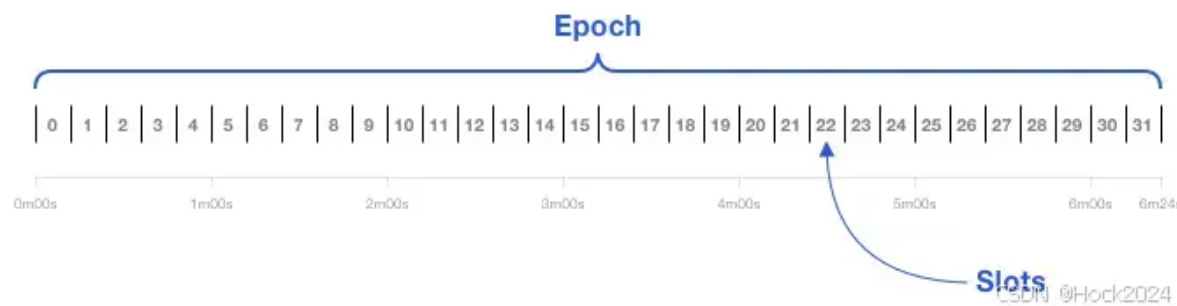
编辑

Gasper

以下主要讨论信标链使用的共识和算法。

以太坊选择的算法叫做Casper FFG，在此基础上，引入另一个规则LMD-GHOST，他们共同组成了信标链的PoS算法Gasper；其中前者是一种投票选择的规定，后者用于分叉情况的处理。（PoS共识算法有很多种，以太坊选择的是Casper FFG）

投票过程



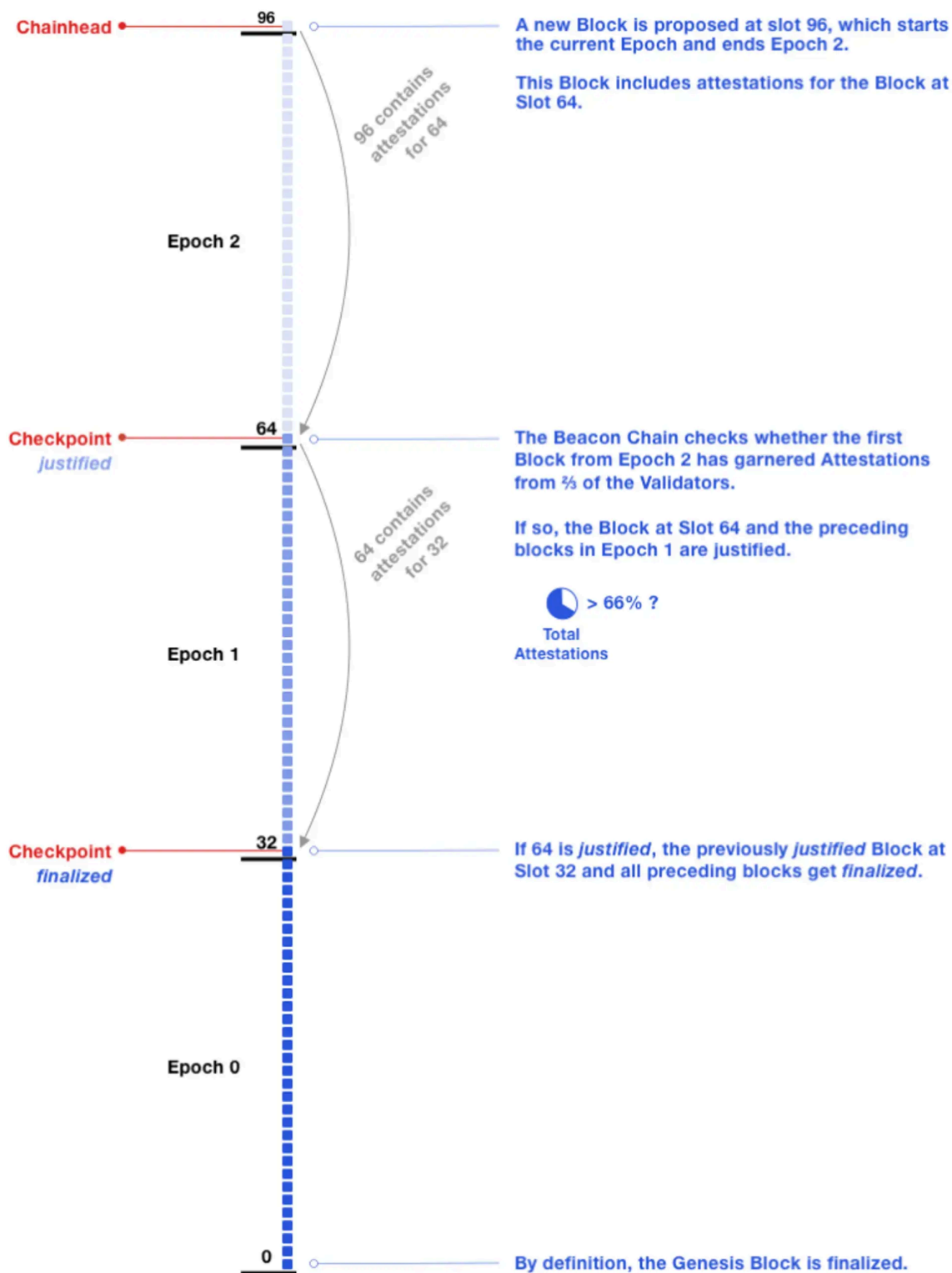
编辑

用户通过质押32个eth成为一名验证者（validator），相当于获得了投票的权利。为了解决节点通信量大的问题，以太坊做了一些时间层次的划分。如图，一个slot（插槽）代表一个出块时间，这个时间是12秒，32个slot组成一个Epoch（纪元），代表一个大周期，时间为6m24s，在这一点上pos的出块稳定程度是高于pow的。接下来，将会随机选择一名验证者，去发起一个区块提议（propose），由它去出块。同时，其它的验证者会组成一个人数 ≥ 128 委员会(committee),委员会通过投票来确认区块，整个过程由一个伪随机算法RANDAO选出（RANDAO算法比较复杂，可以先简单理解为一个黑盒）。

Casper FFG

Casper这种pos算法其实都是bft类型的容错算法，它由pbft共识算法改进而来。因此，我们根据上文“PBFT算法实现流程”可以得到——要实现一轮“视图”，需要进行两轮投票，分别在prepare和commit阶段。这个算法的优点是，只要达成共识，这个节点就是合法存在的（不想BTC里面需要等待6个节点之后确认），链不会出现分叉现象；然而，由于PBFT算法的局限性，我们很难做到在大规模（超多节点）网络中应用这一共识机制。Casper算法就兼容优点并改善了缺点。

在“投票过程”中，我们将一个区块时间定为一个slot（下图中的小块），32个slot就是一个Epoch（一个Epoch作为一个被投票的整体）。最终的敲定分为两个阶段（对于每一个Epoch而言，会接受先后两次投票，两次均通过才为通过）：我们以下图为例进行讲解



CSDN @Hock2024

编辑

上面的链是从下向上挖的，对于Epoch0而言：

1. 当链挖到第32个的时候，大家投票认证 Epoch0，如果这次通过（被2/3以上的节点认可），那么 Epoch0将会从提交状态进入认证状态（justified）
2. 这次投票之后，我们开挖的就是 Epoch1。当达到第64个区块时，进入检查点，节点将对 Epoch0和 Epoch1进行投票（被2/3以上的节点认可），如果通过Epoch1将会从提交状态进入认证状态（justified），而Epoch0将会从认证状态（justified）进入终局性状态（finalized）这时Epoch0就被达成了共识，不需要之后参与讨论了。

下面是一个正在讨论的Epoch

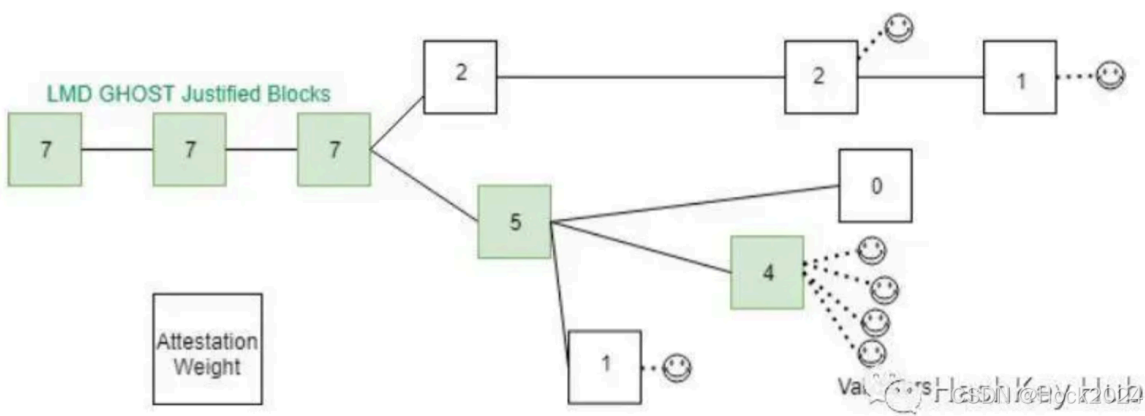
Epoch:	306,179	
Finalized:	No	
Time:	3 mins ago (Aug-23-2024 15:05:59 UTC+8)	TimestampUTCLocal
Attestations:	Calculating...	
Deposits:	Calculating...	
Withdrawals:	224 (5.36259 ETH)	
Slashings P / A:	... / ...	
Voting Participation:	Calculating... (total stake: 33,998,302 ETH)	
Sync Participation:	93.2 % (only counting proposed blocks)	
Validators:	1,062,462	
Avg. Validator Balance:	32.06 ETH	
Slots:	Calculating... (14 Proposed, 1 Pending)	

CSDN @Hock2024

编辑

分叉选择 LMD-GHOST

由于网络延迟或者潜在攻击的问题，新的区块产生可能会发生分叉，这时，我们选择的策略就叫LMD（最新的消息驱动），我们会选择票数最多（权重）的链作为权威链，这一点区别于pow中的最长链原则。下图的一个笑脸代表一个验证者投票，我们选择笑脸最多的链作为合法链，虽然比上面的分叉少一个区块，但它的票数多代表认可多。



编辑

和Casper FFG看上去很相似。Casper FFG是以32个区块为单位进行的整体投票的，是一个单纯的验证方案，而LMD-GHOST算法是在“挖矿”过程中处理区块间分叉的方案。LMD-GHOST处理的是短时间局部分歧，而Casper FFG处理的是长期共识问题。